智慧图书馆数字资源平台

——《数据库管理系统》结课报告

成　员	学　号
程彬彬	202414870203
董梦寒	202414870212
许善颍	202414870328

完成日期：2026 年 11 月

目　　录

一、项目概述…… 3
（一）业务背景…… 3
（二）目标…… 3
（三）分工与进度…… 3
二、技术选型论证…… 4
（一）数据库类型比对…… 4
（二）CAP 取舍…… 5
（三）选型结论…… 6
三、数据模型设计…… 7
（一）逻辑结构…… 7
（二）字段类型…… 7
（三）索引设计…… 9
（四）设计决策…… 9
四、功能实现…… 10
（一）检索…… 10
（二）统计…… 11
（三）索引优化…… 13
（四）权限…… 14
（五）应用程序…… 15
五、文件存储…… 16
（一）GridFS 存储结构…… 16
（二）上传下载实现…… 17
六、架构演进…… 19
（一）副本集部署与故障转移…… 19
（二）分片集群与分片键论证…… 22
七、MongoDB Compass 图形化验收…… 25
（一）业务集合数据量核对…… 25
（二）聚合管道统计核对…… 26
八、问题与反思…… 26
（一）遇到的问题及解决…… 26
（二）收获…… 27
（三）不足与改进…… 27
附　录…… 29
附录 A　图目录…… 29
附录 B　表目录…… 30

一、项目概述

（一）业务背景

传统图书馆纸质资源管理效率低，数字资源（电子书、期刊、读者数据）快速增长，需要一个可扩展的数字资源管理平台。MongoDB 的文档模型适合图书、读者、借阅等半结构化数据，且支持副本集高可用和分片水平扩展。

（二）目标

使用 MongoDB 实现智慧图书馆数字资源平台，完成数据建模、CRUD、索引优化、权限控制、Python 应用集成 GridFS 存储电子书、副本集高可用和分片集群水平扩展。

（三）分工与进度

1. 任务分工

表 1　任务分工

任务	负责人
需求分析与技术选型	程彬彬
数据建模与索引设计	程彬彬、许善颍
单机 MongoDB 实现与权限配置	程彬彬
Python 应用程序开发与 GridFS 集成	董梦寒
副本集部署与故障转移验证	程彬彬、董梦寒
分片集群部署与查询性能测试	许善颍
文档撰写与截图整理	许善颍、董梦寒

2. 进度安排

第 1—2 周完成技术选型与数据建模，第 3—4 周完成单机实现与 Python 应用，第 5 周完成副本集与分片集群，第 6 周完成验收文档。各阶段按“先单机验证、再集群演进”的顺序推进，前一步的集合设计与索引清单直接作为后一步的输入，避免重复建模。

二、技术选型论证

（一）数据库类型比对

1. 关系型数据库与 NoSQL 数据库的差异

本平台面临三个核心问题：馆藏结构字段差异大、借阅流水增长快、电子书文件无处存放。原关系型系统在这三方面都出现瓶颈，根本原因在于关系模型与馆藏数据的“半结构化”特性不匹配。

以本平台 books 集合为例：部分图书含 tags 数组字段，部分图书没有；location 是“楼层一架位一复本数”的嵌套结构；ebook_name 仅部分电子书图书才有。在关系模型中，这类字段差异只能通过两种方式处理：一是为 tags、location 单独建关联表（book_tags、book_location），查询时多次 JOIN；二是全部建成固定列，缺失字段以 NULL 填充。前者使单条图书查询要拆成 3～4 张表，后者导致大量空列和难以维护的宽表。

借阅流水方面，关系型数据库面对高吞吐写入时需要复杂的分库分表方案，而本项目 3000 条流水只是教学样例，真实场景下流水增长以月百万计。电子书文件则根本无法以结构化字段表达，关系型数据库通常只能存文件路径或 BLOB，无法与文档数据自然融合。两者的差异归纳如表 2。

表 2　关系型数据库与 NoSQL 文档数据库区别

对比维度	关系型数据库	NoSQL 文档数据库
数据模型	固定表结构，行一列，需事先定义 Schema	文档（BSON），字段自由嵌套，Schema 灵活
扩展方式	垂直扩展为主，分库分表复杂	水平扩展，副本集与分片集群原生支持
事务支持	强 ACID 事务，成熟完善	支持多文档事务，但更强调单文档原子性
一致性	强一致性	可配置，副本集下默认主节点强一致
字段差异处理	NULL 列或拆表 JOIN	缺字段即省略，天然适应数据不齐
大文件存储	需外挂文件系统或 BLOB	GridFS 原生存储，与文档统一管理

2. 四类 NoSQL 数据库的适用场景对比

NoSQL 数据库按存储模型可分为四类，各自适用场景差异明显，对比如表3。

表 3　四类 NoSQL 数据库的使用场景

类型	代表产品	适用场景	不适用场景
键值型	Redis、Memcached	缓存、会话、计数器，高并发简单读写	多字段组合查询、聚合统计
文档型	MongoDB、CouchDB	半结构化数据、嵌套文档、灵活查询、聚合分析	强关系复杂 JOIN 场景
列式	HBase、Cassandra	海量数据、时序、宽表分析、高吞吐写入	数据模型频繁变化
图型	Neo4j	社交关系、推荐网络、路径分析	常规业务增删改查

本项目选择文档型数据库的理由：本平台的图书数据是典型的半结构化嵌套文档（location 嵌套、tags 数组、字段不齐），恰好是文档模型的天然表达方式；检索需求多样（按类别与价格筛选、按标签多选、模糊检索、按年份范围检索），文档数据库的查询语法和索引能力可以全部覆盖；借阅统计需要分类统计、排行、年度趋势，文档数据库的聚合管道（Aggregation Pipeline）可一站式完成。键值型无法支撑组合查询，列式不擅长度量字段频繁变化，图型则与本平台的业务结构无关——故文档型是唯一同时满足“数据结构、查询、统计”三方面需求的选型。

（二）CAP 取舍

CAP 理论指出，分布式系统在一致性（Consistency）、可用性（Availability）、分区容错性（Partition tolerance）三者中最多同时保证两个。本平台两个核心场景的取舍如下。

场景一：馆藏查询。读者查询图书信息、馆藏位置时，读到过期或错误的数据会直接误导借阅决策（如显示有库存实际已被借出）。该场景优先保证 C（一致性）与 P（分区容错性），即宁可查询短暂不可用，也不返回不一致数据。MongoDB 副本集默认读写都走主节点，正是这种 CP 取向的体现。

场景二：借阅流水写入。读者借还书是高频、即时操作，系统必须时刻接受写入，不能因节点间同步延迟而拒绝读者。该场景优先保证 A（可用性）与 P

（分区容错性），写入先落主节点并返回成功，再异步同步到从节点，允许短暂的不一致窗口。这也是 MongoDB 默认写关注（WriteConcern）策略的设计依据。

BASE 理论的适用性分析：BASE（Basically Available、Soft state、Eventually consistent，即基本可用、软状态、最终一致）适用于可接受短暂不一致的功能。本平台中可以接受的是各类统计报表（学院借阅总量、分类藏书统计、年度趋势），统计结果滞后几秒不影响任何决策；不可接受的是借还书状态与库存扣减，若读者借书时读到“在架”但实际已被借出，将直接造成业务错误。因此平台对实时性要求高的操作走主节点强一致路径，对统计类操作允许从节点读取。

（三）选型结论

因为馆藏数据是字段不齐、天然嵌套的半结构化文档，且平台需要灵活的组合查询与聚合统计、流水数据需要高吞吐水平扩展，所以选择文档数据库 MongoDB 作为统一数字资源平台的存储底座。选型同时伴随两项需要事前化解的风险。

1. 风险一：多集合联合分析能力弱于 SQL

借阅排行、学院统计需要跨 books、readers、borrow_logs 关联，文档数据库的 $lookup 性能与灵活性不如 SQL JOIN。应对思路：在建模阶段合理设计嵌套与冗余字段（如 books 中冗余 borrow_count），高频关联提前物化到统计集合，避免运行时大量 $lookup。

2. 风险二：多文档事务与强一致有性能代价

复杂业务一旦需要跨集合事务，性能开销明显高于关系型。应对思路：将核心业务设计为单文档原子操作（借书即对单条流水插入加对图书字段更新），必要时用补偿机制代替长事务，仅在关键路径使用事务。

三、数据模型设计

（一）逻辑结构

library 数据库由四个普通集合与 GridFS 存储区构成，各集合职责单一、边界清晰，结构如下。

```
library 数据库
├── books          馆藏图书：书目 + 馆藏位置（嵌套）+ 电子书关联
├── readers        读者：证号、学院、等级、偏好类别
├── borrow_logs    借阅流水：借还记录，用于统计与分片测试
├── audit_logs     操作日志：新增/删除/导出等关键操作
└── GridFS 存储区
    ├── fs.files   文件元数据（书名、ISBN、大小、上传时间）
    └── fs.chunks  文件数据块（电子书 PDF 与封面图）
```

（二）字段类型

1. books（馆藏图书）

books 以 isbn 为唯一标识，书目字段与馆藏位置字段同文档存放，字段构成如表 4。

表 4　books 集合字段统计

字段	BSON 类型	说明
isbn	String	ISBN-13，唯一标识
title / author / publisher	String	文本
category	String	6 类之一
price / rating	Double	定价、评分
stock	Int32	在架库存
tags	Array<String>	标签，可缺失
publish_date	Date	出版日期
borrow_count	Int32	历史借阅次数
location	Embedded Document	floor / shelf / copies
ebook_name	String	电子书文件名，可缺失

2. readers（读者）

readers 以借阅证号为主键，记录读者身份、联系方式与借阅偏好，字段构成如表 5。

表 5　readers 集合字段统计

字段	BSON 类型	说明
card_no	String	借阅证号，唯一
name / college	String	姓名、学院
email	String/null	可缺失
phone	String	电话
level	String	普通 / 高级 / 教师
register_date	Date	注册时间
fav_categories	Array<String>	偏好类别，可缺失

3. borrow_logs（借阅流水）

borrow_logs 一条记录对应一次借还，以读者卡号与 ISBN 关联另外两个集合，字段构成如表 6。

表 6　borrow_logs 集合字段统计

字段	BSON 类型	说明
borrow_date	Date	借阅时间
return_date	Date/null	未归还不含此字段
reader_card	String	关联 readers.card_no
book_isbn	String	关联 books.isbn
status	String	已归还 / 借出 / 逾期
duration_days	Int32	借阅时长（天）

4. 易选错字段的类型理由

（1）price 用 Double 而非 String

价格需要参与比较（按价格区间筛选）和聚合（求平均价格），String 只能做文本匹配，无法计算；Double 可直接用于范围查询与 $avg。

（2）日期一律用 Date 而非字符串

源数据虽是 ISO 字符串，但转成 Date 后才支持年份范围查询、按年月 $dateToString 聚合趋势；字符串日期无法高效区间检索。

（3）tags 与 fav_categories 用数组而非逗号拼接字符串

数组支持“同时含多个标签”的 $all 匹配和单标签的 $in 查询，拼接字符串只能做包含模糊匹配，无法精确多选。

（4）stock、borrow_count、duration_days 用 Int32

计数类字段参与加减和排序，整数类型节省空间且运算正确，避免浮点误差。

（三）索引设计

索引覆盖唯一性约束、组合筛选、模糊检索、多键匹配与排序五类需求，清单如表 7。

表 7　索引设计清单

集合	索引字段	类型	用途
books	isbn	唯一索引	主键级唯一性约束、按 ISBN 精确查询
books	category + price	复合索引	类别与价格区间组合筛选
books	title	文本索引	按书名模糊检索
books	publish_date	普通索引	按出版年份范围检索
books	tags	多键索引	标签多选匹配
books	rating	普通索引	按评分排序取前 N
readers	card_no	唯一索引	证号唯一、按证号关联查询
borrow_logs	book_isbn + borrow_date	复合索引	图书借阅排行、年度趋势
borrow_logs	reader_card	普通索引	按读者统计借阅量
audit_logs	create_time	普通索引（倒序）	按时间倒序查最近 100 条

（四）设计决策

1. location 用嵌套而非独立集合

馆藏位置总是随图书一起被查询、一起更新，且位置本身没有独立查询和统计需求；嵌套可将“一本书在哪”一次性返回，避免每次 JOIN。反之，如果将来多校区各自独立管理位置、需要按位置单独统计复本时，就应拆为独立集合。

2. audit_logs 用普通集合

日志结构简单、写入量可控，普通集合配合时间倒序索引即可满足“最近 100 条”查询；若担心日志无限增长，可加 TTL 索引自动过期，不必使用 Capped Collection。

3. borrow_logs 用普通集合加复合索引

流水需要按 ISBN、读者、时间多种维度查询与聚合，普通集合索引策略灵活，且支持后续分片（第六章按分片键切分）；关键参数是 book_isbn + borrow_date 复合索引与按 book_isbn 的排行聚合。不用 Capped 或时间序列集合，因为流水需要长期保留并做跨年趋势统计。

4. 在 books 中冗余 borrow_count 字段

借阅排行时直接按该字段排序，避免每次全表扫描聚合流水，是“以空间换时间”的典型冗余设计。

5. 电子书用 ebook_name 字符串关联 GridFS

书目与文件分离存储，books 只存文件名，文件实体放 GridFS，既避免单文档过大，又支持按书名查书目后定位文件。

6. 分类统计结果物化到 category_stats

统计结果单独落集合，后续查询与导出直接复用，避免重复计算。

四、功能实现

（一）检索

平台在单机 MongoDB 上实现了任务书要求的 6 项检索功能，覆盖组合筛选、多值匹配、模糊查询、范围查询与数据质量核查，结果条数均通过 count_documents() 统计。检索语句与结果如表 8。

表 8　图书检索功能测试结果

序号	检索功能	实现方式	结果条数
1	类别 + 价格区间组合筛选	{category, price:{$gte,$lte}}	8 条
2	标签多选匹配（同时含多个标签）	{tags:{$all:[...]}}	1 条
3	书名模糊检索	{title:{$regex}}	2 条
4	出版年份范围检索	{publish_date:{$gte,$lt}}	16 条
5	缺少 tags 字段核查	{tags:{$exists:false}}	4 条
6	按评分排序取前 10（投影书名、作者、评分）	.sort("rating",-1).limit(10)	10 条

第 5 项验证了任务书中“部分图书无 tags 字段”的数据不齐情况，用于馆

藏数据质量核查；第 6 项通过投影仅返回必要字段，降低传输开销。评分最高的图书为《活着》（余华，4.9 分）。



图 1　图书检索功能运行结果

（二）统计

使用聚合管道（Aggregation Pipeline）实现 5 项统计功能。

1. 按类别统计藏书量、平均价格、平均评分

以 category 分组，配合 $sum、$avg 计算三类指标，结果如表 9。

表 9　各类别图书统计结果

类别	藏书量	平均价格	平均评分
计算机	7 册	78.4	4.5
文学	5 册	61.7	4.7
经济管理	3 册	92.0	4.2
数学	3 册	43.0	4.7
外语	2 册	85.0	4.3

2. 各学院借阅总量前 5

按 readers.college 分组统计借阅次数并降序截取前 5：计算机学院 17 次、文学院 13 次、数学与统计学院 12 次、管理学院 8 次、外国语学院 8 次。

3. 按年月统计借阅趋势

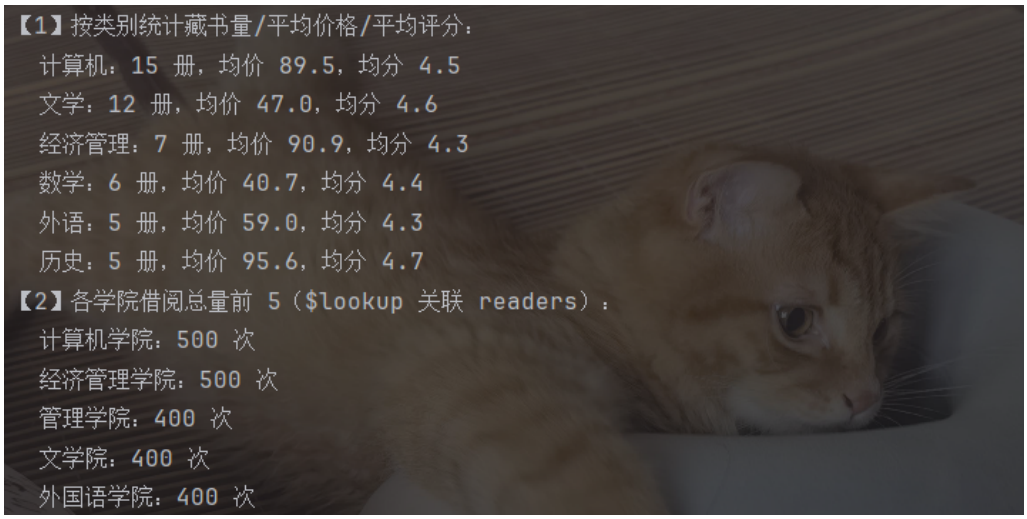
以 $dateToString 将 borrow_date 格式化为“年—月”后分组聚合，覆盖 2023—2025 年，例如 2023-07 为 4 次、2024-07 为 4 次，输出可读的年份与月份标签。

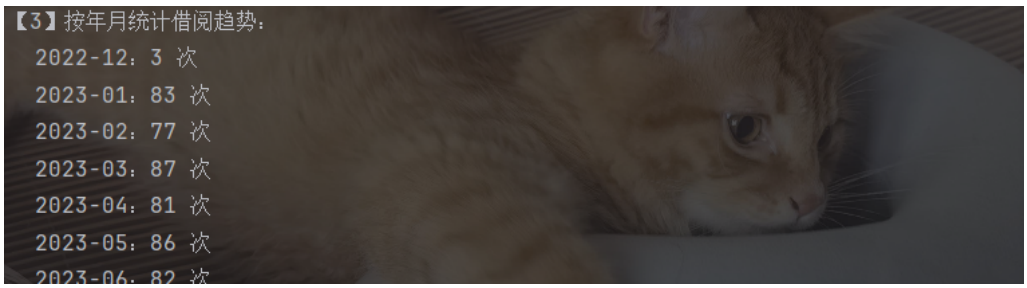
4. 借阅次数最多的 10 本图书并关联馆藏位置

按 borrow_count 降序取前 10，读取嵌套的 location 字段（楼层、架位、复本数），实现“哪本书在哪”一次返回。

5. 分类统计结果写入 category_stats 集合

统计结果经 $project 清洗后通过 $out 落盘为新集合，供后续查询与导出复用，避免重复计算。统计功能的实际运行结果如图 2。





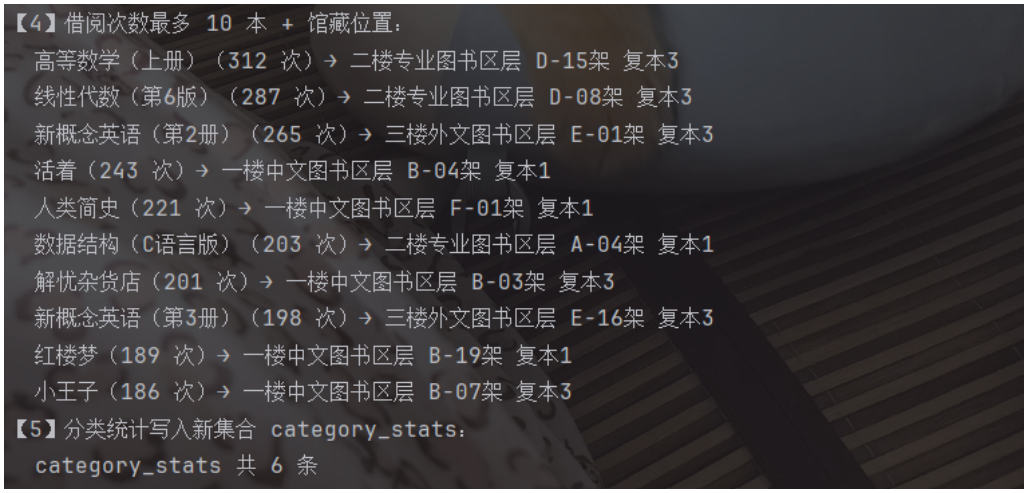


图 2　图书统计分析运行结果

（三）索引优化

按第三章的索引清单创建索引，并选取两条典型查询对比建索引前后的执行计划（explain("executionStats")），结果如表 10。

表 10　索引优化前后性能对比

查询	建索引前耗时 / 扫描文档	建索引后耗时 / 扫描文档 / 索引键
A：类别 + 价格组合筛选	0 ms / 扫描 50 文档	12 ms / 扫描 8 文档 / 查 8 键
B：按 ISBN 查借阅流水	1 ms / 扫描 3000 文档	12 ms / 扫描 60 文档 / 查 60 键

对比可见：查询 A 的扫描文档数从 50 降至 8，查询 B 从 3000 降至 60，执行路径由全表扫描（COLLSCAN）变为索引扫描（IXSCAN），说明索引显著缩小了查询范围。由于教学数据集规模较小，毫秒级耗时差异不明显属正常现象，核心证据是扫描文档数的大幅下降。

同时按清单创建了全部索引：books 的 isbn（唯一）、category + price 复合、title 文本、publish_date、tags 多键、rating；readers 的 card_no（唯一）；borrow_logs 的 book_isbn + borrow_date 复合、reader_card；audit_logs 的 create_time 倒序。数据导入结果如图 3。

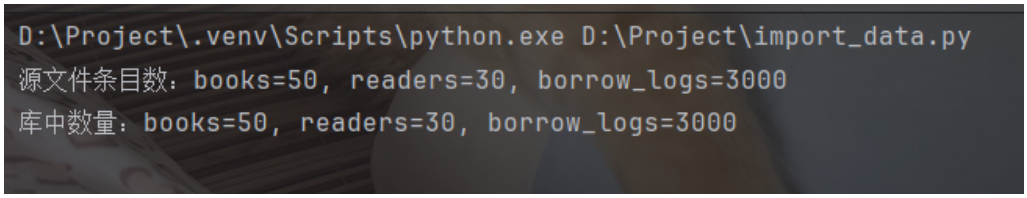


图 3　数据导入结果

（四）权限

为保证图书馆数据安全，采用 MongoDB 内置角色创建三类账号，通过 mongod --auth 启用认证模式，账号设计如表 11。

表 11　数据库账号与权限设计

账号	密码	角色	权限范围
admin	admin123	root（@admin）	数据库超级管理员
rw_user	rw123	readWrite（@library）	可对图书、读者、借阅流水增删改查
ro_user	ro123	read（@library）	仅可查询，不可写入

验证结果：错误密码连接时抛出 AuthenticationFailed，证明认证生效；ro_user 执行 insert 时返回 not authorized，符合只读预期；rw_user 执行 insert 和 delete 均成功，符合读写预期。账号创建与权限验证的截图分别如图 4、图 5。

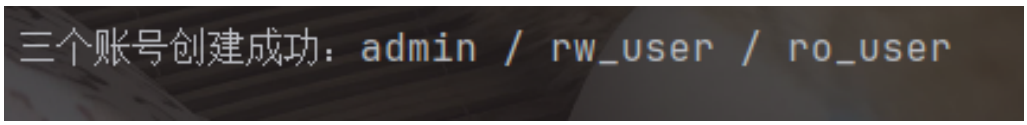


图 4　账号成功创建

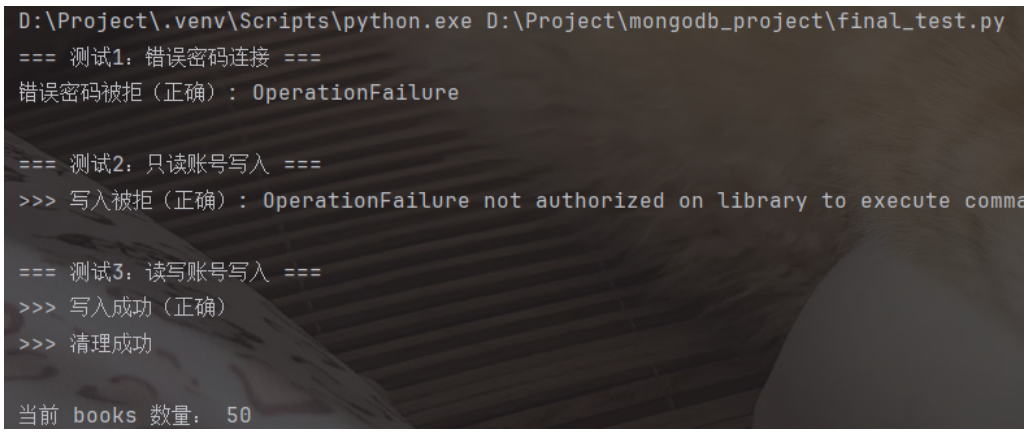


图 5　权限验证结果

（五）应用程序

1. Python 应用程序设计

在 Windows 与 PyCharm 环境下，使用 Python 虚拟环境，通过 pip 安装 pymongo 驱动连接 MongoDB。采用模块化设计，将数据库能力封装为独立可运行的菜单式程序，共 7 个源文件。

db.py：统一数据库连接模块，封装 MongoClient 连接，后续所有模块通过 get_db() 获取数据库对象，避免重复建立连接；同时提供 audit() 函数，所有写操作自动记录日志。

books.py：图书管理，实现按书名、ISBN、类别多条件检索，以及新增图书、删除图书。

readers.py：读者管理，实现按卡号、姓名查询读者，以及新增读者。

borrow.py：借阅管理，实现借书登记、还书登记，自动记录借出和归还时间。

stats.py：统计分析，实现各类别图书数量与均价统计、热门图书 Top5、借阅总览。

gridfs_utils.py：GridFS 文件存储模块，实现上传、列表、下载校验。

main.py：主程序，提供菜单式交互，共 13 个功能项。

2. 运行界面

主程序运行后显示菜单，用户输入数字选择功能，操作完成后返回菜单循环。主菜单界面如图 6。

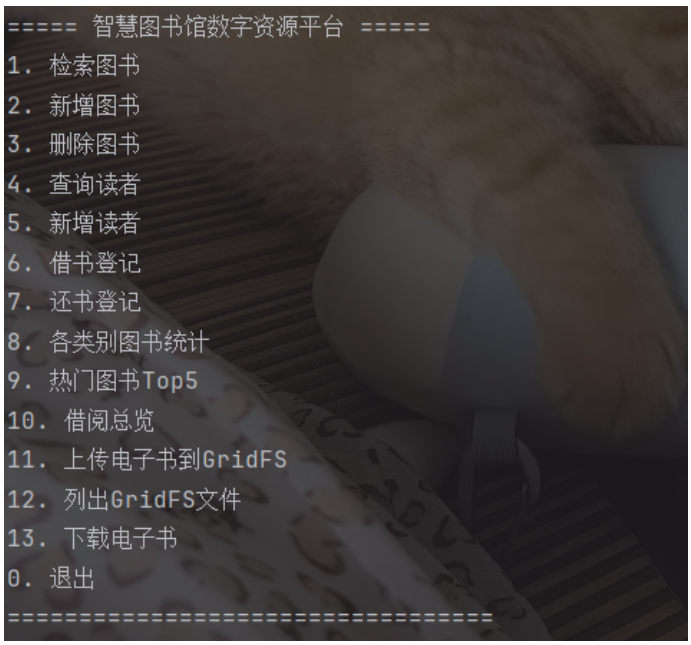


图 6　应用程序主菜单界面

选择统计功能后，各类别图书数量与均价统计结果如图 7。

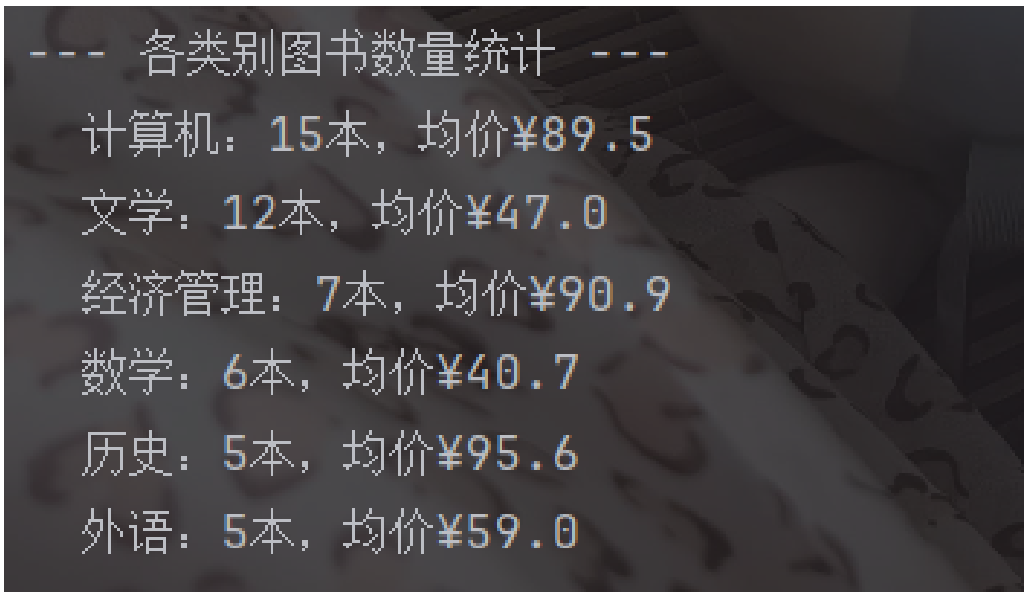


图 7　各类别图书数量统计

五、文件存储

（一）GridFS 存储结构

图书馆平台需要存储电子书 PDF 和封面图片，单文件可达数百 KB 至数 MB。MongoDB 的普通 BSON 文档有 16MB 大小限制，且不适合流式读取大

文件，因此采用 GridFS 规范进行文件存储。

GridFS 将文件分块存储，底层使用两个集合。fs.files 存储文件元数据，每个文档对应一个文件，包含 filename（原始文件名）、length（文件大小）、uploadDate（上传时间），以及自定义元数据 isbn、file_type（pdf 或 cover）、title（书名）。fs.chunks 存储文件内容分块，默认每块 255KB，每个文档包含 files_id（关联 fs.files 的 _id）、n（块序号）和 data（二进制内容）。

上传时，GridFS 将文件按 255KB 切片写入 chunks 集合，最后在 files 集合中写入元数据记录。下载时根据 files._id 找到所有 chunks，按 n 排序拼接还原完整文件。

（二）上传下载实现

1. 文件上传

将 3 个 PDF 电子书和 3 张封面图上传至 GridFS，每个文件通过 ISBN 与 library.books 集合关联，元数据包含文件类型（pdf 或 cover）和原始文件名。上传结果如图 8。

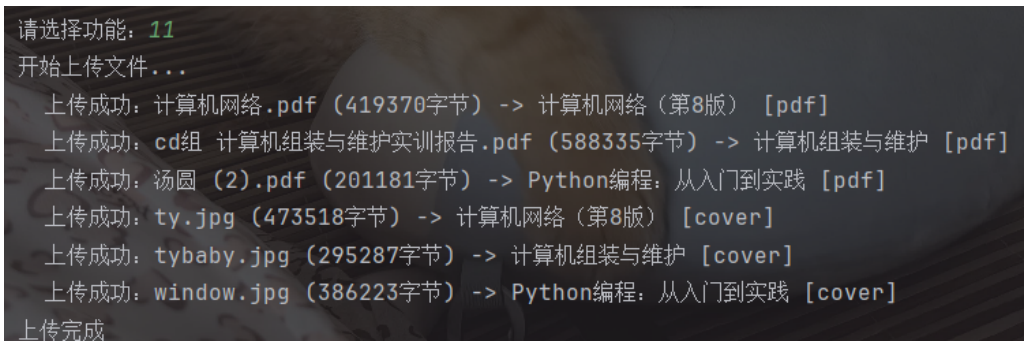


图 8　GridFS 文件上传结果

2. 文件列表

调用 fs.find() 遍历 fs.files 集合，可查看所有文件的文件名、大小、上传时间和关联书名，如图 9。

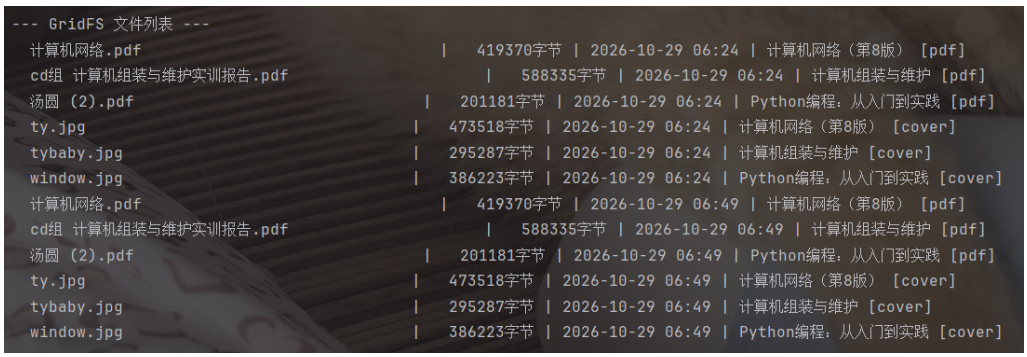


图 9　GridFS 文件列表

3. 下载与校验

按书名关键词检索电子书，用户选择后将文件内容写入本地磁盘。下载完成后，自动比对上传时字节数与下载后本地文件大小。本次下载《计算机网络（第 8 版)》时，上传 419370 字节、下载 419370 字节，校验一致，如图 10。



图 10　GridFS 文件下载与校验结果

4. 操作日志

所有新增图书、删除图书、新增读者、借书、还书、GridFS 上传、下载、统计导出操作，均通过 audit() 函数写入 audit_logs 集合，记录操作类型、详细信息和时间戳，便于后续审计与追踪。

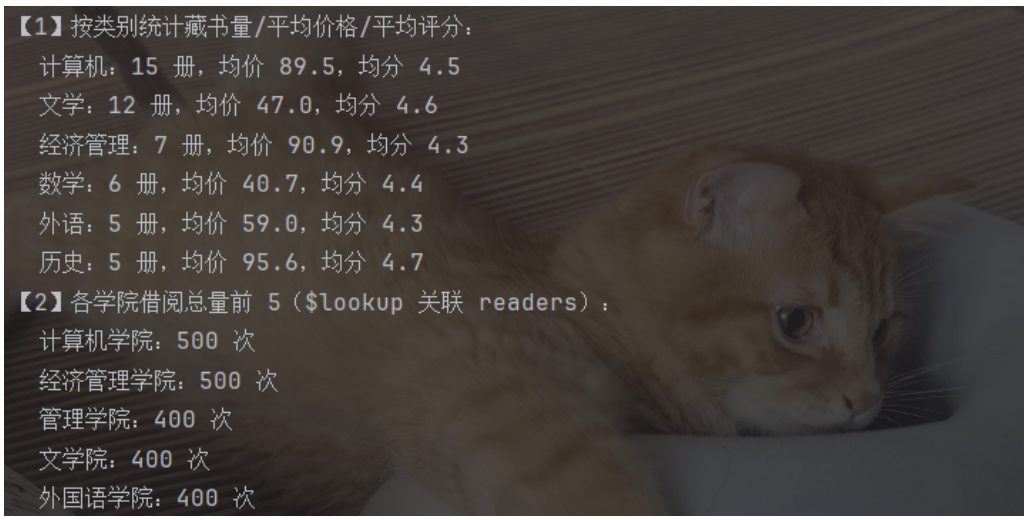


图 12　审计日志记录（Compass 视图）

六、架构演进

（一）副本集部署与故障转移

1. 节点规划与启动

在单机上通过不同端口和数据目录模拟三个节点，配置如表 12。

表 12　副本集节点配置

节点	端口	数据目录	角色
节点 1	27017	D:\mongodb\rs1	主节点（初始）
节点 2	27018	D:\mongodb\rs2	从节点
节点 3	27019	D:\mongodb\rs3	从节点（被动节点）

三个节点均以 mongod --replSet rs0 参数启动，连接 27017 执行 rs.initiate() 初始化副本集，再通过 rs.add() 添加另外两个节点。初始化完成后，27017 被选举为主节点（PRIMARY），27018 和 27019 为从节点（SECONDARY），如图 12。

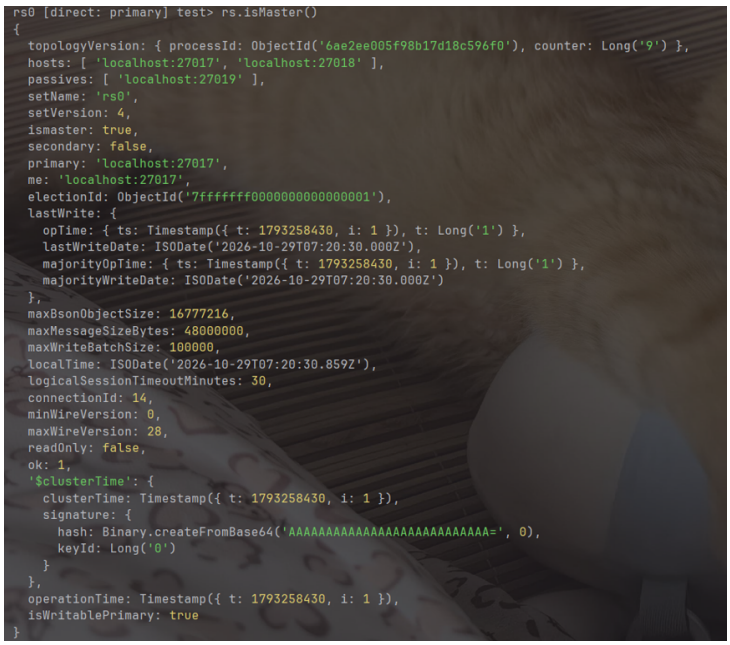


图 12　副本集初始状态（27017 为 PRIMARY）

2. 数据读写与读取偏好

在主节点写入测试数据后，验证两种读取偏好。primary 指所有读请求都路由到主节点，适用于要求读取最新数据、不能容忍旧数据的场景（如图书库存实时查询）；secondaryPreferred 指优先从从节点读、主节点不可用时自动降级，适用于读多写少、可接受短暂延迟的场景（如图书检索、排行榜）。

从节点默认不允许读取，需通过 setReadPref() 显式配置读取偏好。两种模式均成功读到同一条测试记录，如图 13。



图 13　读取偏好验证（primary 与 secondaryPreferred）

3. 故障转移演练

手动停止主节点 27017（Ctrl+C），副本集在几秒内自动选举 27018 为新的主节点。通过 rs.isMaster() 确认 primary 已切换为 localhost:27018，isWritablePrimary 为 true，如图 14。

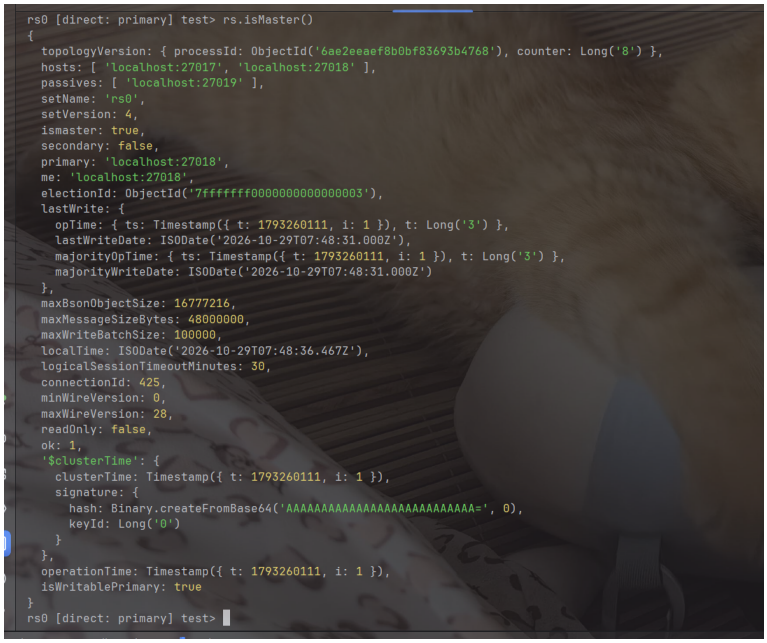


图 14　故障转移后状态（27018 为新 PRIMARY）

随后重启 27017，它自动以从节点身份重新加入副本集，不影响当前主节点运行。故障转移后查询之前写入的测试数据，数据一致无丢失，验证了副本集的数据冗余和高可用能力。

4. 节点增删演练

通过 rs.remove("localhost:27019") 移除节点，再通过 rs.add("localhost:27019") 重新添加。副本集配置版本号递增，节点重新同步数据后恢复正常状态，整个过程不影响主节点读写，如图 15。

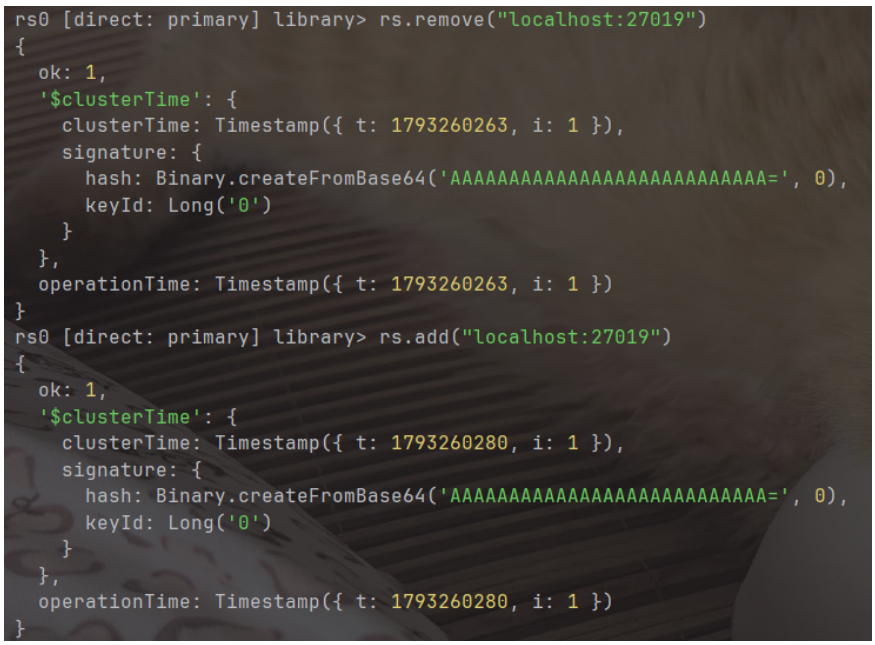


图 15　节点动态增删操作（rs.remove 与 rs.add）

（二）分片集群与分片键论证

1. 集群拓扑

当借阅流水持续增长，单机或副本集的存储与写入能力达到上限时，需要水平扩展。本任务部署了三节点分片集群，拓扑如表 13。

表 13　分片集群节点配置

角色	端口	副本集	数据目录
配置服务器	27020	cfgrs	D:\mongodb\cfg
分片 1	27021	shard1rs	D:\mongodb\shard1
分片 2	27022	shard2rs	D:\mongodb\shard2
mongos 路由	27023	—	—

配置服务器以单节点副本集启动，两个分片各为单节点副本集，mongos 作为查询入口。通过 sh.addShard() 将两个分片加入集群，sh.enableSharding("library") 启用数据库分片，sh.shardCollection() 对 borrow_logs 集合按 reader_card 启用分片。

2. 分片键选择论证

选择 {reader_card: 1} 作为 borrow_logs 的分片键，理由有四点。

（1）基数高。reader_card 为读者卡号，本项目有 30 个读者，基数足够，

数据分布均匀。

（2）常用查询条件。按读者查询借阅记录是高频操作，等值查询可直接路由到单个分片，避免广播。

（3）数据分布均匀。不同读者的借阅记录数量相近，按 reader_card 分片不会产生热点。

（4）备选方案对比。若选择 {borrow_date: 1}，虽然时间范围查询方便，但同一时间段的借阅记录集中在一个 chunk，容易导致写入热点，且按读者查询时需要广播到所有分片。因此选择 reader_card 更优。

3. 数据分布验证

导入 3000 条 borrow_logs 后，通过 splitAt 和 moveChunk 将数据均衡分布到两个分片。getShardDistribution 显示 shard1rs 和 shard2rs 各存储约 50% 的文档，如图 16。



图 16　分片数据分布结果（各约 50%）

定向查询：按分片键 reader_card="R016" 查询，执行计划为 SINGLE_SHARD，仅路由到 shard2rs 一个分片，利用 reader_card 索引做 IXSCAN，如图 17。

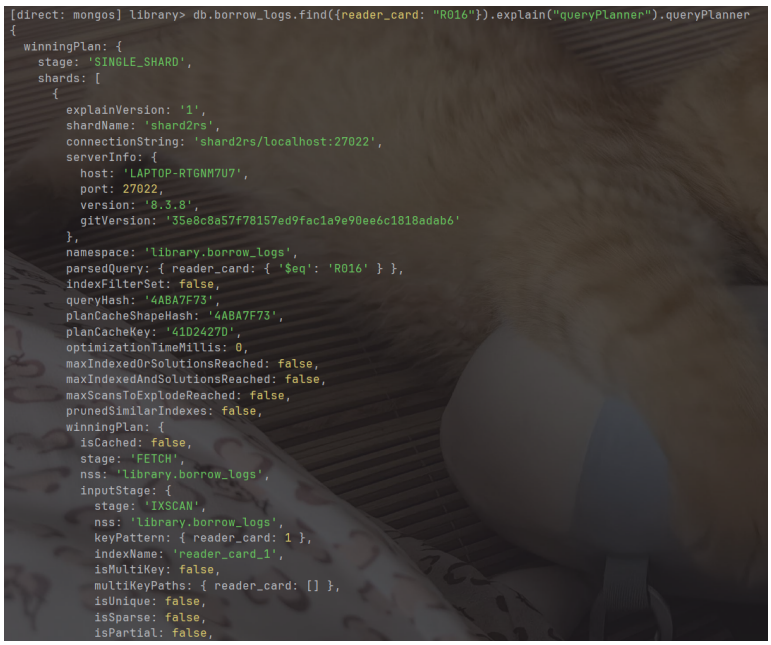


图 17　定向查询执行计划（SINGLE_SHARD）

4. 跨片查询与定向查询对比

跨片查询：按非分片键 borrow_date 查询，执行计划为 SHARD_MERGE，需要同时访问两个分片并合并结果，如图 18；实际访问的分片数为 2 个，如图 19。

```
[direct: mongos] library> db.borrow_logs.find({borrow_date: {"$gte": "2024-01-01
"}}).explain("queryPlanner").queryPlanner.winningPlan.stage
SHARD_MERGE
```

图 18　跨片查询执行计划类型（SHARD_MERGE）

```
[direct: mongos] library> db.borrow_logs.find({borrow_date: {"$gte": "2024-01-01
"}}).explain("queryPlanner").queryPlanner.winningPlan.shards.length
2
```

图 19　跨片查询访问分片数（2 个）

两类查询的差异说明：分片键的价值不在于存储切分本身，而在于让尽可能多的查询只落在单个分片上；一旦查询条件与分片键无关，mongos 就必须广播到全部分片再合并结果，延迟随分片数增加而上升。

5. 分片集群访问控制

分片集群整套环境（mongos 路由、配置服务器副本集、每个分片副本集）统一使用 keyFile 密钥认证。首先在 mongos 路由上创建集群管理员账号；所有组件启动时开启 --auth。客户端连接 mongos 路由必须提供账号密码，禁止无认证直接访问分片、配置服务器节点。

七、MongoDB Compass 图形化验收

（一）业务集合数据量核对

使用 MongoDB Compass 图形客户端连接 library 数据库，对业务集合的数据量逐一核对：books 集合文档共 50 条、readers 集合文档共 30 条、borrow_logs 集合文档共 3000 条，导入数据完整，与原始数据量匹配。

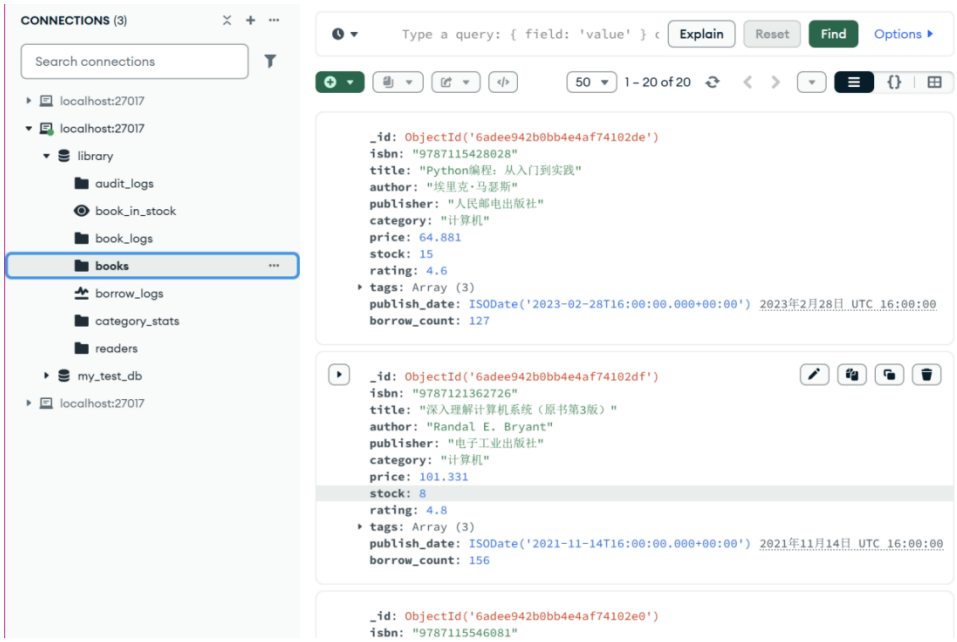


图 20　books 集合文档数量核对

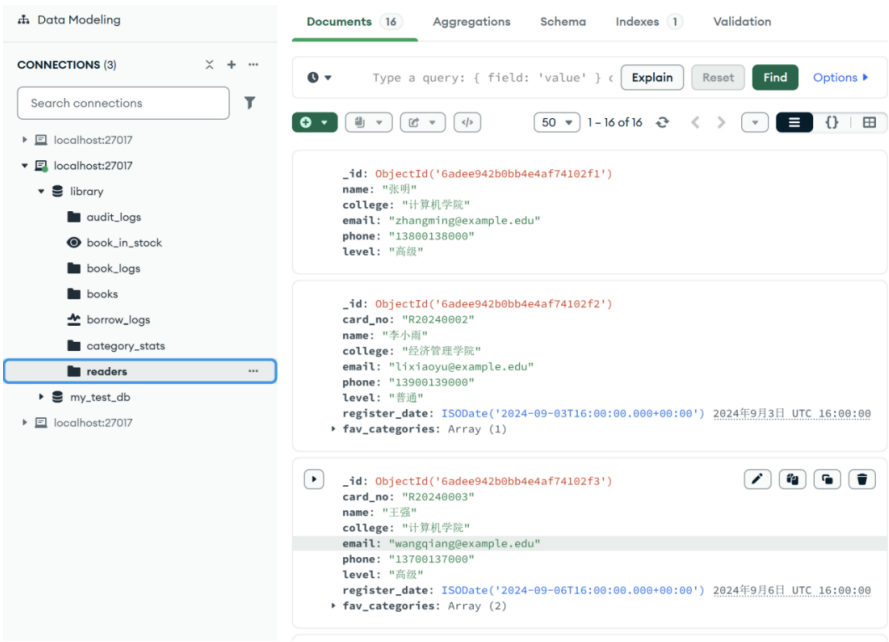


图 21　readers 集合文档数量核对

两个集合的文档数量与任务书给出的基准数据一致，说明批量导入脚本未发生漏写或重复写入；borrow_logs 集合的 3000 条流水同时是后续分片测试的数据基础。

（二）聚合管道统计核对

在 Compass 的聚合面板中录入图书分类统计的聚合管道并执行，得到藏书数量、图书平均价格等统计结果。对比 Shell 命令行、Python 程序的统计输出，图形界面结果与命令行结果一致，验证统计逻辑正确、数据可靠。

图形化验收的意义在于把散落在命令行的结论变成可复核的界面证据：集合数量、文档内容与聚合结果都能在同一客户端内直接查看，为报告中的各项统计结论提供了界面层面的佐证。

八、问题与反思

（一）遇到的问题及解决

1. 副本集与分片启动慢

首次启动副本集节点时，终端持续输出 oplog.rs not found 错误，一度以为启动失败。实际是节点已启动但等待初始化，执行 rs.initiate() 后错误消失。

2. 权限验证反复

ro_user 角色配置正确但仍能写入，排查后发现是残留 mongo 进程占用端口，重新清理所有进程后重启解决。

3. 分片集群配置复杂

分片集群需要配置服务器、分片节点、mongos 三层架构，且每个节点都需先初始化为副本集才能加入集群，初次部署时多次因节点未就绪导致 addShard 失败。

三项问题指向同一个经验：MongoDB 的集群操作大多是有先后依赖的异步过程，报错往往不是配置错误，而是上一步尚未完成。因此部署脚本中需要显式等待节点状态就绪，再执行下一步。

4. 审计日志漏写

早期版本调用借书、还书函数时，忘记调用 audit() 审计函数，操作不会写入 audit_logs 集合。排查时在每个写操作入口埋点打印日志，确认业务分支，最终在全部增删改、上传下载调用处强制调用 audit() 函数，修复后所有业务操作均可生成审计记录。

5. 分片查询路由异常

最开始分片键配置错误，导致按 reader_card 查询依然执行 SHARD_MERGE 全分片广播。排查方式是查看 explain 执行计划，确认索引与分片键保持一致后重建分片集合，问题解决。

（二）收获

通过本项目，完整实践了 MongoDB 从单机到副本集再到分片集群的演进过程：单机模式完成数据建模、索引优化和权限控制；副本集验证了高可用和故障自动转移；分片集群验证了水平扩展能力。理解了分片键选择对查询性能的影响——按分片键查询只访问单分片，非分片键查询需要广播合并。

更具体地，本项目的三组数据构成了对课程知识点的直接印证：索引使查询 A 的扫描文档数由 50 降至 8、查询 B 由 3000 降至 60，说明索引的收益应以扫描量为衡量而非耗时；副本集在主节点停止后数秒内完成选举，说明高可用依赖选举机制而非人工干预；分片后两个分片各占约 50%，且定向查询只访问一个分片，说明分片键同时决定了数据分布与查询路径。

（三）不足与改进

（1）当前分片集群每个分片只有单节点，生产环境应每个分片至少 3 节点保证高可用。

（2）数据量较小（3000 条），分片优势不明显，实际百万级数据时跨片查询延迟差异会更显著。

（3）未实现分片集群下的认证配置，后续应在 mongos 和分片节点上统一启用 --auth 并创建管理账号。

（4）应用层缺少输入过滤防护，存在 NoSQL 注入风险，未对特殊查询操作符做拦截，容易造成隐私数据泄露。

（5）业务未使用 MongoDB 事务包裹借还书流程，存在库存与借阅记录不一致的隐患；GridFS 电子书也缺少下载权限校验。

（6）审计日志仅记录操作内容，未记录客户端 IP，且日志本身可被修改删除，审计追溯能力不足。

（7）系统接口没有访问限流，恶意高开销查询会引发拒绝服务。后续应增加输入校验、事务保护、访问权限与日志加固。

附　录

附录 A　图目录

图 1　图书检索功能运行结果
图 2　图书统计分析运行结果
图 3　数据导入结果
图 4　账号成功创建
图 5　权限验证结果
图 6　应用程序主菜单界面
图 7　各类别图书数量统计
图 8　GridFS 文件上传结果
图 9　GridFS 文件列表
图 10　GridFS 文件下载与校验结果
图 11　审计日志记录（Compass 视图）
图 12　副本集初始状态（27017 为 PRIMARY）
图 13　读取偏好验证（primary 与 secondaryPreferred）
图 14　故障转移后状态（27018 为新 PRIMARY）
图 15　节点动态增删操作（rs.remove 与 rs.add）
图 16　分片数据分布结果（各约 50%）
图 17　定向查询执行计划（SINGLE_SHARD）
图 18　跨片查询执行计划类型（SHARD_MERGE）
图 19　跨片查询访问分片数（2 个）
图 20　books 集合文档数量核对
图 21　readers 集合文档数量核对

附录 B　表目录

表 1　任务分工
表 2　关系型数据库与 NoSQL 文档数据库区别
表 3　四类 NoSQL 数据库的使用场景
表 4　books 集合字段统计
表 5　readers 集合字段统计
表 6　borrow_logs 集合字段统计
表 7　索引设计清单
表 8　图书检索功能测试结果
表 9　各类别图书统计结果
表 10　索引优化前后性能对比
表 11　数据库账号与权限设计
表 12　副本集节点配置
表 13　分片集群节点配置